# Software Design Document

**Selected use case:** UC2 – Find a Match (Strategy, Adapter and Observer patterns)

**Project:** Skill/Interest-Based Secure Matching Platform for Erasmus and Local Students

**Course:** BIL 482 Design Patterns – TOBB University of Economics and Technology, Fall 2026

**Instructor:** Dr. Çiğdem Avcı

**Assignment:** Project Assignment 2 (PA2) – Use Case Implementation with Design Pattern Integration

**Team:** Sangre y Oro – Alberto Alvarez Polo (261118082), Alejandro Blas Piris (261118081), Carlos Molina Perez (261118083)

**Document authors:** Carlos Molina Perez (main author: sections 1–5, 8, 10, 13–18 and integration); Alejandro Blas Piris (sections 7 and 9, design patterns); Alberto Alvarez Polo (sections 6, 11 and 12, architecture, user interface and external interfaces)

**Version:** 1.0 – 6 November 2026 · **Source code:** src/ in the team repository github.com/903155/bil482-erasmus-matching-platform

# Table of Contents

**1. Introduction . . . . 4**

1.1 Purpose and Selected Use Case . . . . 4

1.2 Scope of the Implementation . . . . 4

1.3 Traceability Conventions . . . . 4

**2. System Overview . . . . 4**

**3. System Context . . . . 5**

**4. Key Features and Functionality . . . . 5**

**5. Assumptions and Dependencies . . . . 6**

5.1 Assumptions . . . . 6

5.2 Dependencies . . . . 6

**6. Architectural Design . . . . 6**

6.1 System Architecture Diagram (High-Level) . . . . 6

6.2 Architectural Patterns and Styles . . . . 7

6.3 Rationale for Architectural Decisions . . . . 7

**7. Component Design . . . . 8**

7.1 Subsystems and Modules . . . . 8

7.2 Responsibilities of Each Component . . . . 8

7.3 Interfaces Between Components . . . . 9

7.4 Component Diagram . . . . 9

7.5 Dynamic Behaviour . . . . 10

**8. Data Design . . . . 11**

8.1 Data Model / ER Diagram . . . . 11

8.2 Data Storage . . . . 11

8.3 Data Flow Diagram . . . . 12

8.4 Data Validation Rules . . . . 12

**9. Design Patterns . . . . 13**

9.1 Applied Design Patterns . . . . 13

9.2 Strategy – Context and Justification . . . . 13

9.3 Observer – Context and Justification . . . . 14

9.4 Adapter – Context and Justification . . . . 15

9.5 Measured Improvement . . . . 15

**10. Implementation Notes . . . . 16**

**11. User Interface Design . . . . 16**

**12. External Interfaces . . . . . 18**
12.1 APIs . . . . . 18
12.2 Third-party Systems . . . . . 19

**13. Performance Considerations . . . . . 19**
13.1 Performance Requirements . . . . . 19
13.2 Scalability and Optimisation Strategies . . . . . 19

**14. Error Handling and Logging . . . . . 19**
14.1 Exception Management . . . . . 19
14.2 Logging Mechanisms . . . . . 20

**15. Design for Testability . . . . . 20**

**16. Deployment and Installation Design . . . . . 21**
16.1 Environment Configuration . . . . . 21
16.2 Packaging and Dependencies . . . . . 21

**17. Change Log . . . . . 22**

**18. Future Work / Open Issues . . . . . 22**

**19. Task Matrix . . . . . 23**

# 1. Introduction

## 1.1 Purpose and Selected Use Case

This document describes the design of the software implemented for Project Assignment 2. The team selected **UC2 – Find a Match** from the Software Requirements Specification (SRS, PA1) because it is the central function of the platform and because it combines three of the design patterns mapped in PA1: **Strategy** (interchangeable matching criteria), **Adapter** (translation of profile texts, UC2-A2) and **Observer** (notifications of match requests, reused from UC3). It was also the use case presented at the Design Pattern Workshop.

## 1.2 Scope of the Implementation

The implementation covers the complete main flow UC2-S1 to UC2-S9, the alternative flows UC2-A1 to UC2-A3 and the exceptions UC2-E1 to UC2-E4, through all three layers of the architecture: a web interface, a JSON API, the Business Layer with the patterns and a Data Access Layer. Registration (UC1), event management (UC3) and reporting (UC4) are outside this assignment; UC2 reads the profiles created by UC1 from a data file and writes notifications that the UC3 panel would display.

## 1.3 Traceability Conventions

| Identifier | Meaning | Where it appears |
|---|---|---|
| UC2-S1 … S9, UC2-A1 … A3, UC2-E1 … E4 | Steps, alternative flows and exceptions of UC2 (SRS 3.3) | This document, code comments, exception classes, test descriptions |
| FR-2.1 … FR-2.5, NFR-x.y | Functional and non-functional requirements (SRS 3.3, 4) | This document, tests |
| DV-01 … DV-07 | Data validation rules (section 8.4) | Exception flow ids, code comments |
| UC2-T01 … T18 | Automated tests of the selected use case (section 15) | src/test, test report |
| AT-UC2-01 … 03 | Acceptance tests of the SRS traceability matrix (SRS 8) | Tests UC2-T04, T07, T06, T18 |

# 2. System Overview

The Erasmus & Local Student Matching Platform is a multilingual web application (Turkish, English, Spanish) that connects incoming Erasmus students with local students who share interests and language goals. In UC2 a logged-in student chooses a **matching criterion** – Language Exchange, Hobbies or Sports – and optional filters. The system calculates a compatibility score from 0 to 100 for every eligible student, shows the best ten with the reasons for their score, and lets the student send a **match request**. The invited student receives a notification and accepts or rejects it; when it is accepted both students are notified and can see each other's contact details.

The delivered software is a single, self-contained Java application (one JAR file of 75 KB) that embeds the web server, serves the user interface and exposes a REST-style JSON API. It has **no external dependencies**: it needs only a Java 17 runtime.

| Item | Value |
|---|---|
| Language / platform | Java 17 (tested with OpenJDK 21), JDK HttpServer, HTML5 + JavaScript |
| Source size | 47 production classes (1,669 lines), 8 test classes (468 lines), 1 HTML page (246 lines) |
| Tests | 18 automated tests, all passing (section 15) |
| Patterns | Strategy, Adapter, Observer (+ Dependency Injection, Builder, Repository as supporting techniques) |

## 3. System Context

Figure 1 shows UC2 inside its environment. The only human actor is the **Student** (Erasmus or local). Profiles are produced by UC1; in this assignment they are provided as a data file. Match events are passed to the notification panel of UC3, and profile descriptions are translated by an external **Translation API**, which is simulated by a mock, as decided in the SRS (scope, section 1.2).

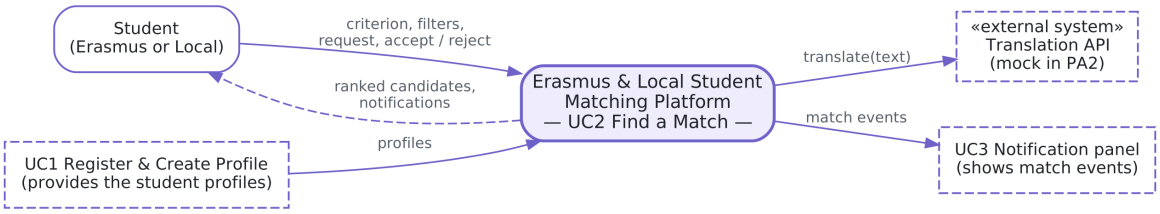



*Figure 1 – System context of UC2 Find a Match*

## 4. Key Features and Functionality

| Feature | Description | SRS | Main classes | Tests |
|---|---|---|---|---|
| Matching criteria | Three interchangeable criteria selected at runtime | UC2-S1, S3, FR-2.1 | MatchingStrategy, MatchingStrategyRegistry | T01–T04, T09 |
| Compatibility score | Score 0–100 with human-readable reasons, results ordered, top 10 | UC2-S5, S6, FR-2.2 | Language/Hobby/SportsMatchingStrategy | T01–T04 |
| Filters | Language, student type (default: opposite type), department, availability | UC2-S2, FR-2.3 | MatchFilter | T14 |
| Exclusion rules | Never shows self, blocked users, users with matching disabled, incomplete profiles, confirmed partners or candidates rejected for the same criterion | UC2-S4, FR-2.5, UC2-A3 | MatchingService.isEligible | T04, T05, T10 |
| Match requests | Send, accept or reject; states PENDING, CONFIRMED, REJECTED | UC2-S7–S9, FR-2.4 | MatchingService, MatchRequest | T07, T08, T10 |
| Notifications | Automatic in-app notifications and audit log for every match event | UC2-S8, S9, FR-3.4 | MatchEventPublisher, observers | T07, T13 |
| Translate description | Shows a profile text in the interface language; falls back to the original | UC2-A2, UC2-E4 | TranslationAdapter, ProfileDescriptionService | T11, T12 |

| Feature | Description | SRS | Main classes | Tests |
|---|---|---|---|---|
| Multilingual UI | Interface in English, Spanish and Turkish, switchable at any time | NFR-U.3 | index.html | manual |

# 5. Assumptions and Dependencies

## 5.1 Assumptions

- Student profiles already exist and are complete or incomplete as created by UC1; UC2 does not edit them.
- Authentication (eduroam, UC1) has already happened. The demo UI lets the reviewer choose the logged-in student from a list instead of logging in.
- The data volume of the pilot is small (hundreds to a few thousand students), so profiles fit in memory.
- Match requests and notifications only need to live while the server runs in this prototype; a database is planned for PA3 (section 18).
- Contact details shown after a match are generated from the student id (demo e-mail addresses).

## 5.2 Dependencies

| Dependency | Type | Notes |
|---|---|---|
| Java SE 17 or newer (JDK for building, JRE for running) | Runtime / build | Only standard library: java.net.http, com.sun.net.httpserver, java.util.logging |
| Web browser (Chrome, Firefox, Edge, Safari) | Client | No front-end framework; plain HTML/CSS/JavaScript |
| data/students.csv | Data | Demo profiles; a copy is packaged inside the JAR as a fallback |
| Translation API | External system | Simulated by MockTranslationApiClient (adaptee) |
| UC1 / UC3 of the platform | Internal | Profiles in, match events out (Figure 1) |

# 6. Architectural Design

## 6.1 System Architecture Diagram (High-Level)

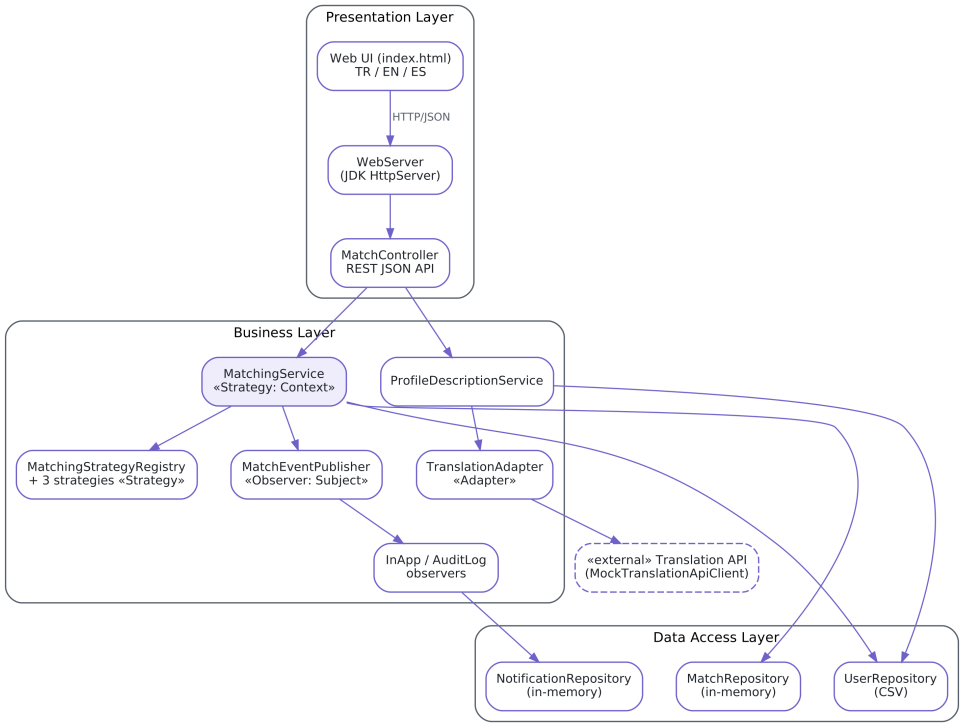



*Figure 2 – Layered architecture of UC2 with its components and patterns*

## 6.2 Architectural Patterns and Styles

- **Layered architecture** (from PA1): Presentation → Business → Data Access. Each layer only calls the layer below it, and only through interfaces for the Data Access Layer (UserRepository, MatchRepository, NotificationRepository).
- **Client–server with a REST-style JSON API**: the browser is a thin client; all rules run on the server. The API (section 12.1) is the contract between the Presentation Layer and the Business Layer.
- **Embedded server, single deployable**: the HTTP server is part of the application, so the whole system is one JAR that starts with one command (section 16).
- **Composition root**: MatchingApplication is the only class that knows the concrete implementations and connects them (manual Dependency Injection).

## 6.3 Rationale for Architectural Decisions

| Decision | Alternatives considered | Reason for the choice |
|---|---|---|
| Keep the Layered architecture of PA1 | MVC framework, microservices | Matches the SRS and PA1; the patterns live in the Business Layer; three people can work on different layers. Microservices would add network and deployment complexity without benefit for one use case. |
| Plain Java, no frameworks or libraries | Spring Boot, Javalin, Maven/Gradle with JUnit | Reviewer can build and run with only a JDK and no internet access; every pattern is visible in our own code instead of being hidden by a framework; small JAR (75 KB). |

| Decision | Alternatives considered | Reason for the choice |
|---|---|---|
| Repository interfaces with CSV / in-memory implementations | Direct SQL from the services, an embedded database | Satisfies NFR-PO.4 (the storage can be replaced by changing only the Data Access Layer) while keeping the prototype dependency-free. The ER model (8.1) is already designed for the database of PA3. |
| Strategy selected per request (local variable), not a mutable field | Classic setStrategy() on a shared service | The service is shared by concurrent HTTP requests; a mutable strategy field would let two users overwrite each other's criterion. Selecting per call keeps the service stateless and thread-safe. |
| Server-rendered JSON + client-side rendering | Server-side HTML templates | Keeps the Business Layer independent of the UI and lets the same API serve a future mobile app. |

# 7. Component Design

## 7.1 Subsystems and Modules

| Package (module) | Layer | Contents |
|---|---|---|
| com.sangreyoro.matching.presentation | Presentation | WebServer, MatchController, StaticFileHandler, HTTP helpers |
| src/main/resources/web | Presentation | index.html – single-page user interface (TR/EN/ES) |
| …matching.business | Business | MatchingService (UC2 use-case service, Strategy Context) |
| …matching.business.strategy | Business | MatchingStrategy, three ConcreteStrategies, registry, SetSimilarity |
| …matching.business.notification | Business | MatchEventPublisher (Subject), MatchObserver, two observers, MatchEvent |
| …matching.business.translation | Business | TranslationService (Target), TranslationAdapter, mock API (Adaptee), ProfileDescriptionService |
| …matching.data | Data Access | Repository interfaces and CSV / in-memory implementations |
| …matching.domain | Shared model | Student, MatchFilter, MatchCandidate, MatchRequest, Notification, enums |
| …matching.exception | Shared | MatchingException hierarchy with SRS flow ids |
| …matching.util | Shared | Json writer, logging configuration |

## 7.2 Responsibilities of Each Component

| Component | Responsibility |
|---|---|
| MatchController | Maps HTTP requests to service calls, validates parameters, converts results to JSON and exceptions to HTTP status codes. Contains no matching logic. |
| MatchingService | Implements UC2: checks the profile (UC2-E1), applies the exclusion rules and filters, delegates scoring to the selected strategy, ranks results, creates and answers match requests, publishes events. |

| Component | Responsibility |
|---|---|
| MatchingStrategy + implementations | Calculate a 0–100 score and the reasons for one pair of students, each with its own algorithm (language complement, Jaccard similarity of hobbies, sports + common time). |
| MatchingStrategyRegistry | Returns the strategy for a criterion; new strategies are registered, never coded into the service. |
| MatchEventPublisher | Keeps the observer list and notifies all observers; isolates failures of one observer. |
| InAppNotificationObserver / MatchAuditLogObserver | Create notification messages / write audit log lines. |
| TranslationAdapter | Converts our translate() call to the external API call and its status codes to our exception model; caches results. |
| ProfileDescriptionService | Returns the profile description in the requested language or the original text with a notice (UC2-E4). |
| Repositories | Load and store students, match requests and notifications; thread-safe. |

## 7.3 Interfaces Between Components

| Interface | Provided by | Used by | Main operations |
|---|---|---|---|
| HTTP API /api/* | MatchController | Web UI (any HTTP client) | see section 12.1 |
| MatchingService (public methods) | MatchingService | MatchController | findMatches(studentId, criterion, filter), sendRequest(from, to, criterion), respond(requestId, who, accept) |
| MatchingStrategy | Concrete strategies | MatchingService (via registry) | criterion(), score(me, other) |
| MatchObserver | Observers | MatchEventPublisher | update(MatchEvent) |
| TranslationService | TranslationAdapter | ProfileDescriptionService | translate(text, from, to) |
| UserRepository / MatchRepository / NotificationRepository | Data Access Layer | Business Layer | findById, findAll / save, findById, findByStudent / save, findByStudent |

## 7.4 Component Diagram

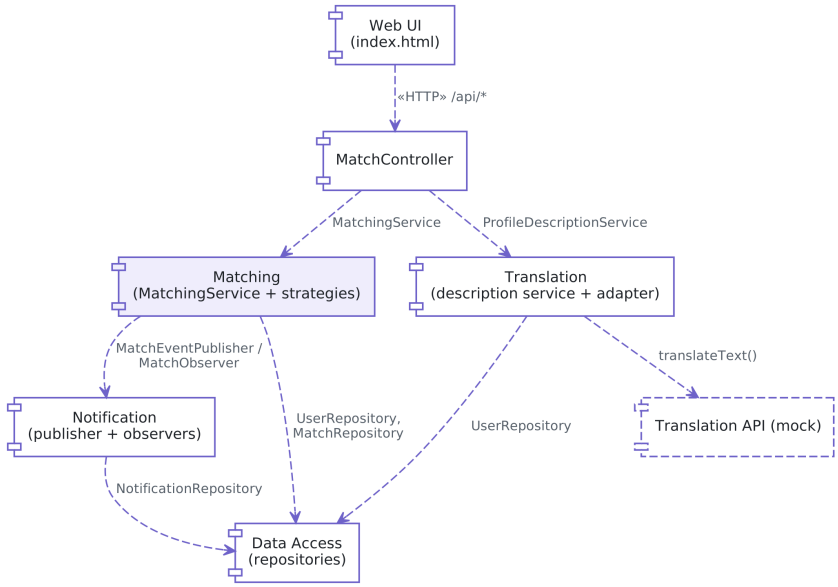



*Figure 3 – Component diagram – dashed arrows show the interface each component requires*

## 7.5 Dynamic Behaviour

Figure 4 shows the main flow UC2-S1 to UC2-S6: the service selects the strategy for the chosen criterion and calls it once per eligible candidate. Figure 5 shows UC2-S7 to UC2-S9 with the Observer notifications, and Figure 6 the life cycle of a match request.

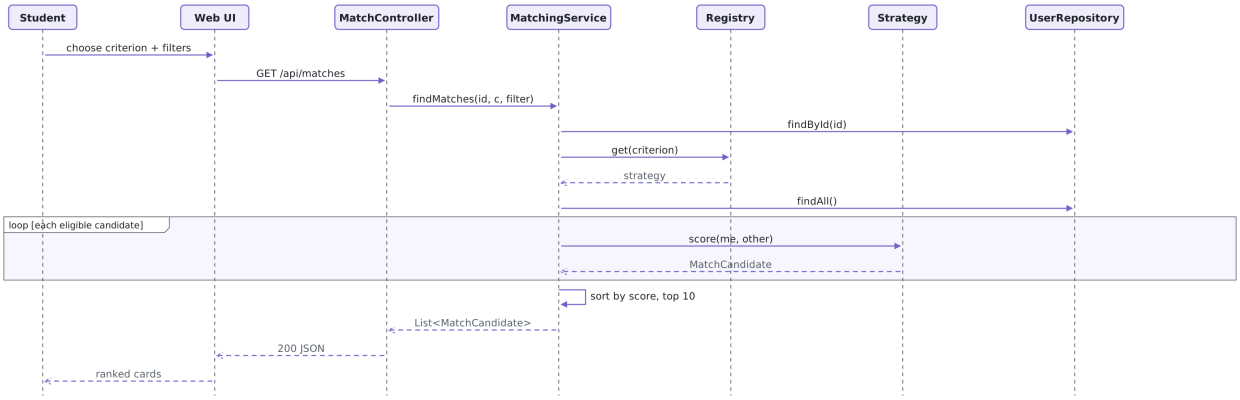



*Figure 4 – Sequence diagram – find matches (UC2-S1 to UC2-S6, Strategy)*

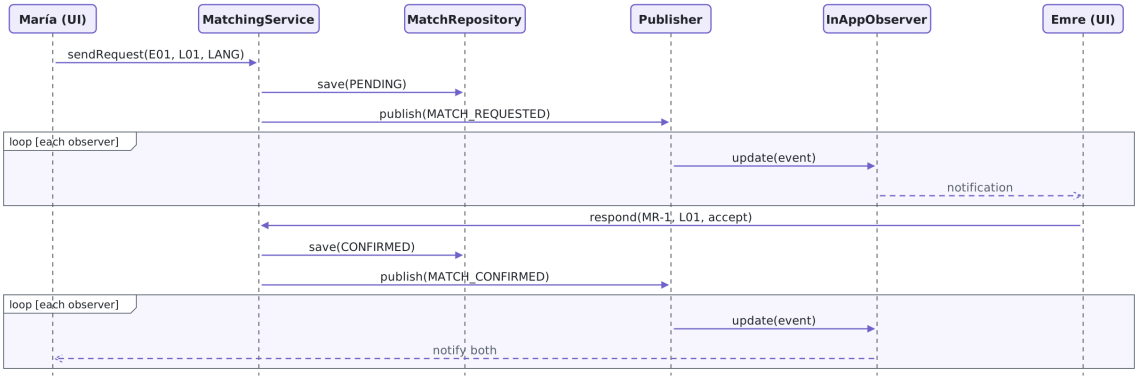



*Figure 5 – Sequence diagram – send and accept a request (UC2-S7 to UC2-S9, Observer)*

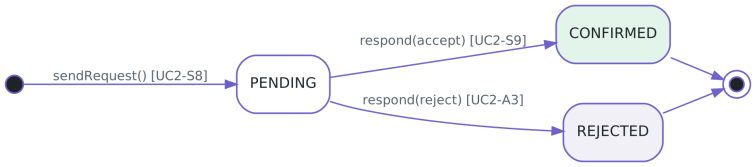



*Figure 6 – State machine of a match request*

# 8. Data Design

## 8.1 Data Model / ER Diagram

Figure 7 is the logical data model of UC2. It is the target relational schema for PA3; the current prototype keeps the same entities as Java objects (Student, MatchRequest, Notification), with the multi-valued attributes (languages, hobbies, sports, availability, blocked users) stored as sets inside Student.

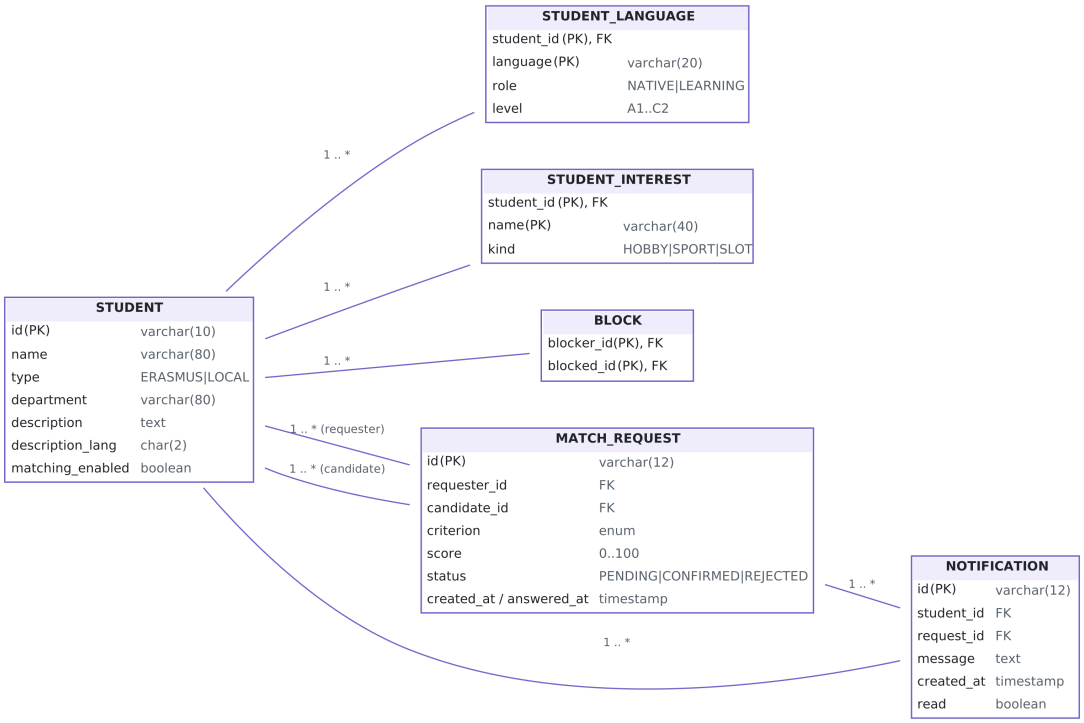



*Figure 7 – ER diagram of the UC2 data*

## 8.2 Data Storage

| Data | Storage in PA2 | Format / structure | Planned (PA3) |
|---|---|---|---|
| Student profiles | data/students.csv, loaded at start-up into an unmodifiable map (O(1) look-up by id) | 13 fields separated by ';', lists separated by ','; a packaged copy in the JAR is the fallback | STUDENT, STUDENT_LANGUAGE, STUDENT_INTEREST, BLOCK tables |
| Match requests | InMemoryMatchRepository (ConcurrentHashMap) | MatchRequest objects, ids MR-1, MR-2, … | MATCH_REQUEST table |

| Data | Storage in PA2 | Format / structure | Planned (PA3) |
|---|---|---|---|
| Notifications | InMemoryNotificationRepository (per student, CopyOnWriteArrayList) | Notification objects, ids N-1, N-2, … | NOTIFICATION table |
| Translations | Cache inside TranslationAdapter (ConcurrentHashMap) | key = from\|to\|text | unchanged |
| Logs | logs/matching-0.log … matching-4.log (rotating, 1 MB each) | one line per record (section 14.2) | unchanged |

*Example line of students.csv:*

```
L01;Emre Yılmaz;LOCAL;Computer Engineering;turkish;spanish:B1,english:B2;
photography,hiking,programming;football,running;weekday-evenings,weekends;true;;
tr;Fotoğraf çekmeyi ve doğa yürüyüşlerini seviyorum.
```

## 8.3 Data Flow Diagram

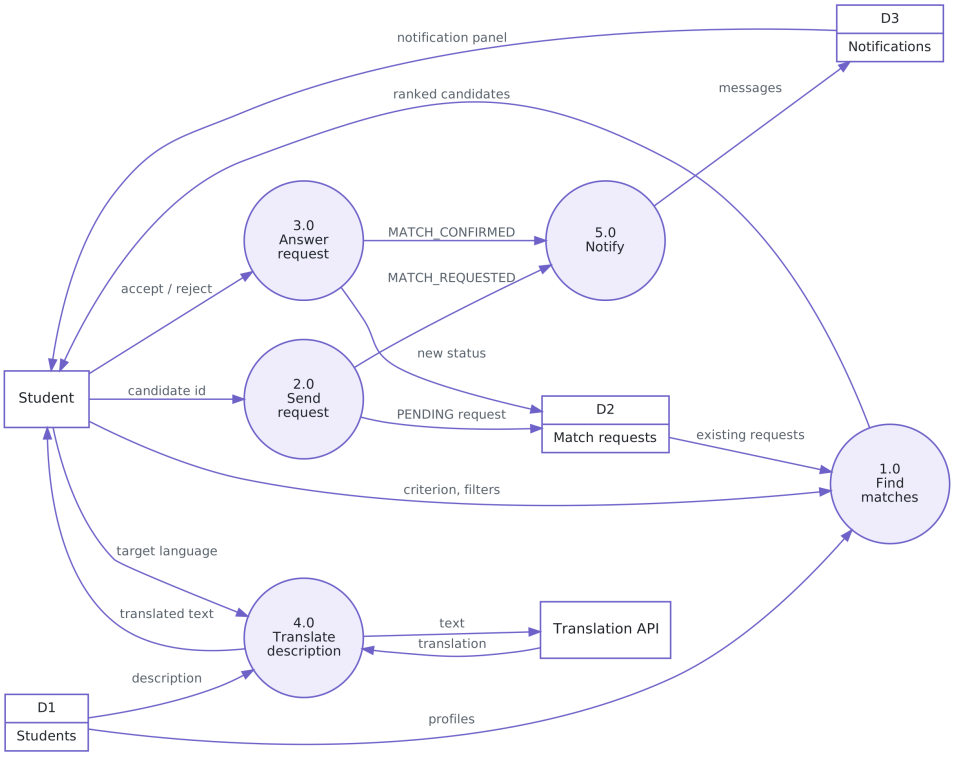



*Figure 8 – Data flow diagram (level 1) of UC2*

## 8.4 Data Validation Rules

| Rule | Rule description | Where it is enforced | Result when broken |
|---|---|---|---|
| DV-01 | Every student id in a request must exist | MatchingService.requireStudent | 404, StudentNotFoundException |
| DV-02 | A student cannot send a request to themselves | MatchingService.sendRequest | 400, InvalidMatchRequestException |
| DV-03 | No request to or from a blocked user or a user with matching disabled | MatchingService.sendRequest | 400, InvalidMatchRequestException |

| Rule | Rule description | Where it is enforced | Result when broken |
|---|---|---|---|
| DV-04 | Only the invited student answers a request, and only once | MatchingService.respond | 400, InvalidMatchRequestException |
| DV-05 | Required parameters present; criterion, type and level are valid enum values; filters trimmed and compared case-insensitively | MatchController, MatchFilter, enums | 400 with error code DV-05 |
| DV-06 | Each data line has 13 fields, valid type and CEFR level, unique id; a bad line is skipped and logged | CsvUserRepository | line skipped, WARNING in log |
| DV-07 | Request bodies are limited to 16 KB; static files cannot be read outside /web | HTTP helpers, StaticFileHandler | body truncated / 400 |
| UC2-E1 | Searching requires a complete profile (≥ 1 hobby or sport, ≥ 1 native and ≥ 1 learning language) | Student.isProfileComplete | 409, IncompleteProfileException |
| UC2-E3 | Only one open (pending or confirmed) request per pair of students, in both directions | MatchingService.sendRequest | 409, DuplicateRequestException |

# 9. Design Patterns

## 9.1 Applied Design Patterns

| Pattern | Category | Roles → classes | UC2 steps |
|---|---|---|---|
| Strategy | Behavioral | Context: MatchingService · Strategy: MatchingStrategy · ConcreteStrategies: LanguageExchangeStrategy, HobbyMatchingStrategy, SportsMatchingStrategy · selection: MatchingStrategyRegistry | UC2-S1, S3, S5 |
| Adapter | Structural | Target: TranslationService · Adapter: TranslationAdapter · Adaptee: MockTranslationApiClient · Client: ProfileDescriptionService | UC2-A2, UC2-E4 |
| Observer | Behavioral | Subject: MatchEventPublisher · Observer: MatchObserver · ConcreteObservers: InAppNotificationObserver, MatchAuditLogObserver · event: MatchEvent (push model) | UC2-S8, S9, A3 |
| Supporting techniques | – | Dependency Injection (constructor injection, composition root MatchingApplication), Builder (Student.Builder), Repository (data interfaces) | all |

## 9.2 Strategy – Context and Justification

**Context.** UC2 lets the student choose *how* to be matched (FR-2.1). The three criteria compute the score with completely different algorithms, the user changes the criterion at runtime (UC2-A1), and new criteria are expected ("same degree", "music taste").

**Justification.** Strategy encapsulates each algorithm in its own class behind one interface, so the service never contains criterion-specific code. An if/else or switch inside MatchingService would grow

with every criterion, violate the Open/Closed Principle and force re-testing of the whole service after each change. Template Method was rejected because the algorithms do not share a common skeleton and the choice must change at runtime, not by inheritance.

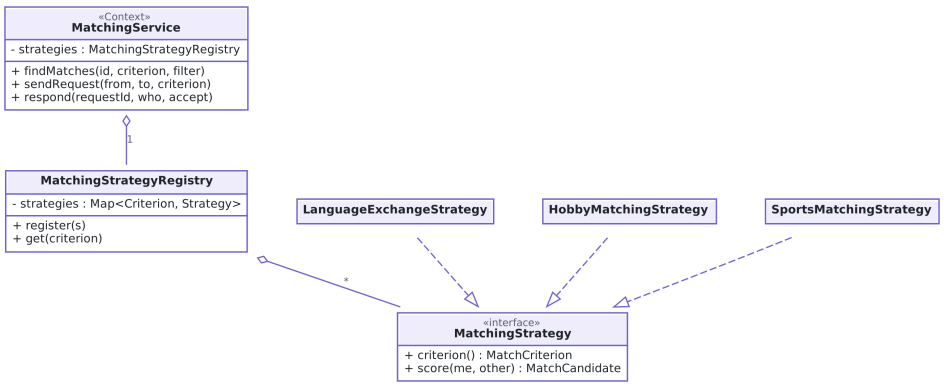



*Figure 9 – Strategy pattern in UC2*

```
MatchingStrategy strategy = strategies.get(criterion);          // UC2-S3: choose the strategy
for (Student other : users.findAll()) {                          // UC2-S4: candidate profiles
    if (!isEligible(me, other, filter, criterion, myRequests)) continue;
    MatchCandidate c = strategy.score(me, other);                // UC2-S5: delegate to the strategy
    if (c.score() > 0) result.add(c);
}
```

## 9.3 Observer – Context and Justification

**Context.** When a request is created, confirmed or rejected, several receivers must react: the invited student's notification panel, the requester's panel and the audit log; e-mail or push notifications are planned.

**Justification.** With Observer, MatchingService publishes one event and does not know the receivers. New channels attach themselves without changing the service. The publisher catches the exception of a failing observer, so an outage of one channel cannot lose the in-app notification (NFR-R.4). A direct call from the service to each channel was rejected because it couples the use case to every channel.

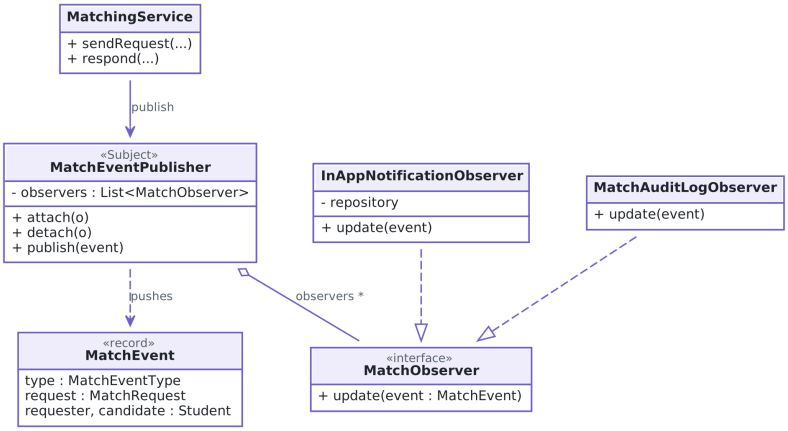



*Figure 10 – Observer pattern in UC2*

## 9.4 Adapter – Context and Justification

**Context.** Profile descriptions are written in the author's language. The external translation API has an interface we cannot change: another method name, parameters in another order, upper-case language codes and HTTP-like status codes instead of exceptions. In PA2 it is a mock that will later be replaced by a real provider.

**Justification.** The adapter implements our TranslationService interface and translates both the call and the error model, so the rest of the system never depends on the external API. Changing the provider means writing one new adapter. A Facade was not chosen because we do not simplify a subsystem of many classes; we convert one incompatible interface into the one our client expects.

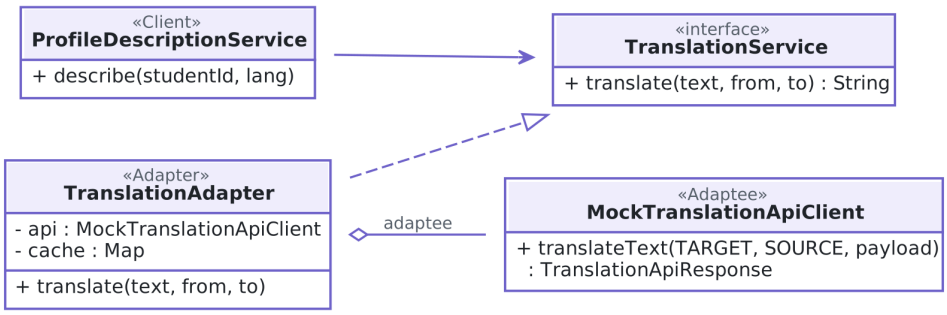



*Figure 11 – Adapter pattern in UC2*

## 9.5 Measured Improvement

The assignment asks for measurable metrics where possible. The values below were measured on the delivered code and are checked by the automated tests where indicated.

| Pattern | Metric | Value | Evidence |
|---|---|---|---|
| Strategy | Lines changed in MatchingService to add or replace a criterion | 0 | Test UC2-T09 plugs in a new strategy without touching the service |
| Strategy | Concrete strategy classes referenced by MatchingService | 0 (only the interface) | grep of the source |
| Strategy | Cyclomatic complexity of findMatches | 6, constant (if/else version: 6 + 1 per criterion) | code review |
| Observer | Notification channel classes referenced by MatchingService | 0 (1 publisher) | grep of the source |
| Observer | Events delivered when another observer fails | 100 % | Test UC2-T13 |
| Adapter | Classes that depend on the external API | 1 adapter (+ composition root) | grep of the source |
| Adapter | External calls for a repeated translation | 0 (cache) | Test UC2-T11 checks the call counter |
| Overall | Average findMatches time with 1,000 students | ≈ 2.2 ms (limit NFR-P.2: 3,000 ms) | Test UC2-T16 |

Every place where a pattern is applied carries a header comment in the source code with the pattern name, its intent, the motivation, the improvement and these metrics (MatchingStrategy, MatchEventPublisher, TranslationAdapter; shorter comments in each participant class).

## 10. Implementation Notes

- **Project layout:** standard Maven-style folders without Maven: *src/main/java*, *src/main/resources/web*, *src/test/java*, *data/*; build scripts *build.sh* (Linux/macOS) and *build.bat* (Windows).
- **Traceability in code:** comments name the SRS step they implement (e.g. *// UC2-S5: delegate to the strategy*); each business exception carries its flow id (UC2-E1, UC2-E3, UC2-E4, DV-xx) which also appears in the HTTP error response and in the log; test methods carry the id UC2-Txx and the items they cover.
- **Immutability:** Student, MatchFilter and MatchCandidate are immutable, so strategies cannot change a profile; MatchRequest changes state only through confirm() and reject().
- **Concurrency:** the web server uses a fixed thread pool; repositories use concurrent collections, the observer list is copy-on-write and the service is stateless (strategy chosen per call).
- **Privacy:** contact details are returned only for CONFIRMED matches; a rejected requester is not told who rejected (UC2-A3); JSON output escapes '<' so user text cannot inject HTML.
- **Deviation from PA1:** the Observer for UC2 notifications was mapped to UC3 in PA1; it is implemented here because UC2-S8/S9 need it, and UC3 will reuse the same MatchObserver mechanism. A MatchingStrategyRegistry was added to select strategies without a switch statement.

## 11. User Interface Design

The interface is a single page with three columns: **criterion and filters** (left), **ranked compatible students** (centre) and **notifications and requests** (right). Each result card shows the score as a number and a bar, the reasons for the score, interests, the description with a *Translate* button and the *Send match request* button. The page switches between English, Spanish and Turkish at any time without losing the current search (NFR-U.3), every function is at most two clicks away (NFR-U.4), and notifications refresh every 5 seconds (NFR-P.3). The screenshots below are taken from the running application.

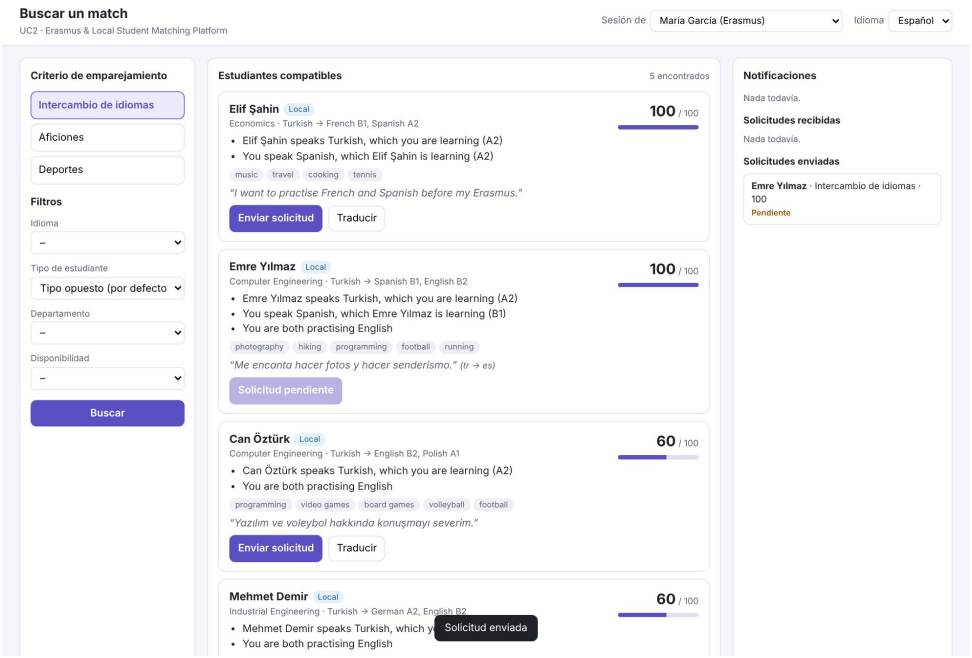



*Figure 12 – María (Erasmus) sees her language partners in the Spanish interface, translated Emre's Turkish description and sent him a request*

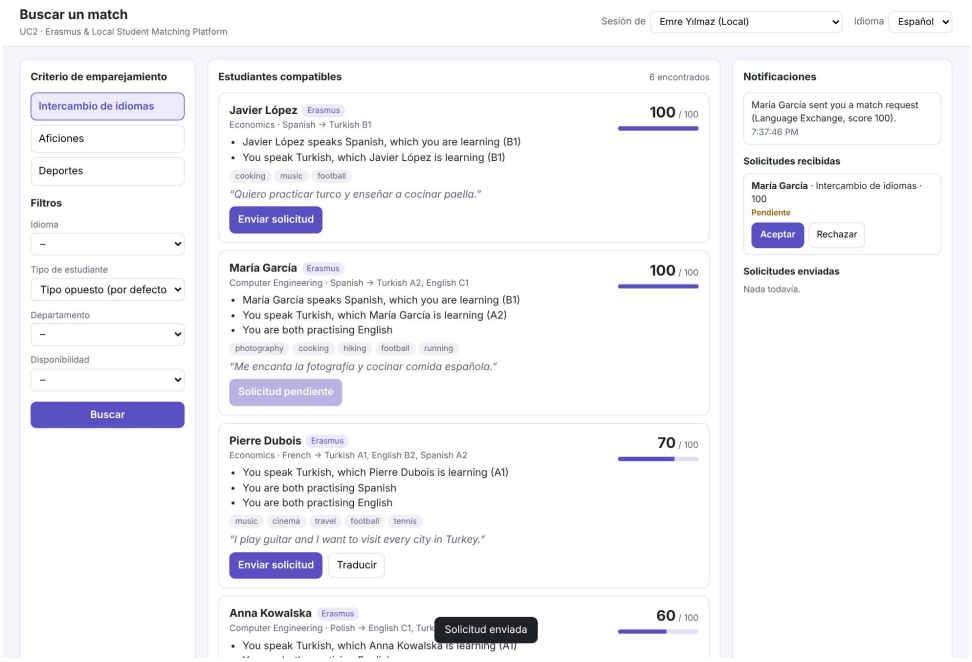



*Figure 13 – Emre (local) receives the notification and can accept or reject the request*

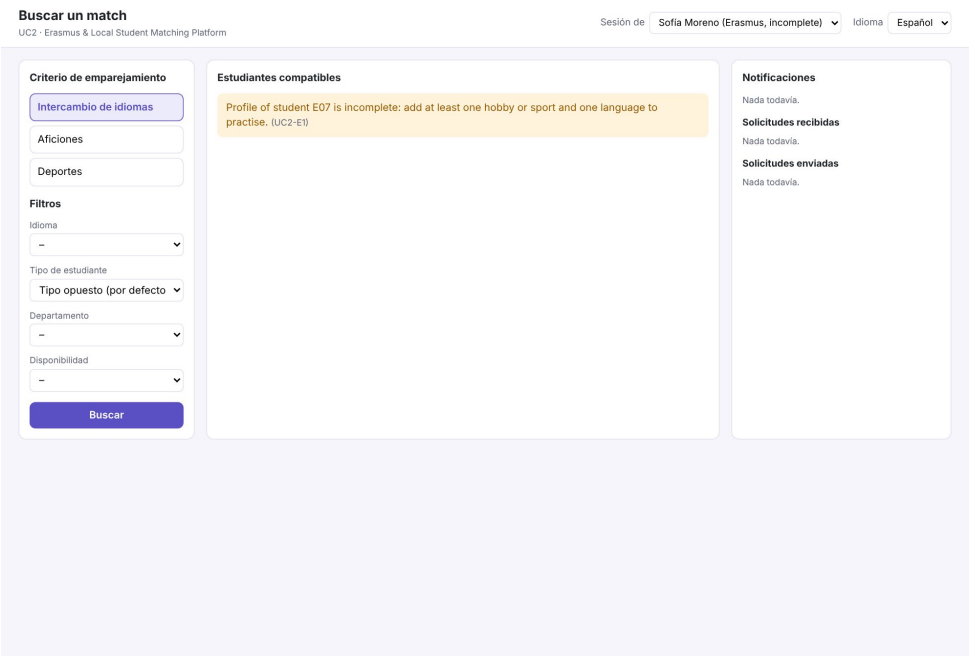



*Figure 14 – UC2-E1 – a student with an incomplete profile gets an explanatory message*

# 12. External Interfaces

## 12.1 APIs

REST-style JSON API provided by MatchController (base URL http://localhost:8080). POST bodies are application/x-www-form-urlencoded. Errors return *{"error": "<flow id>", "message": "..."}*.

| Method and path | Parameters | Result | UC2 |
|---|---|---|---|
| GET /api/students | – | 200: list of profiles | demo login |
| GET /api/criteria | – | 200: available criteria | S1 |
| GET /api/matches | studentId, criterion; optional language, type, department, availability | 200: {criterion, count, candidates[{..., score, reasons, requestStatus}], message?} · 409 UC2-E1 | S2–S6, E1, E2 |
| POST /api/requests | requesterId, candidateId, criterion | 201: request (PENDING) · 409 UC2-E3 · 400 DV-02/03 | S7, S8, E3 |
| POST /api/requests/{id}/accept \| reject | studentId | 200: request (CONFIRMED / REJECTED) · 400 DV-04 | S9, A3 |
| GET /api/requests | studentId | 200: sent and received requests | S8, S9 |
| GET /api/notifications | studentId | 200: notifications, newest first | S8, S9 |
| GET /api/students/{id}/description | lang (en, es, tr) | 200: {text, language, translated, notice} | A2, E4 |

### 12.2 Third-party Systems

| System | Interface | Status in PA2 | Integration |
|---|---|---|---|
| Translation API | translateText(TARGET, SOURCE, payload) → (statusCode, translatedPayload, errorMessage) | Mock with the translations of the demo descriptions; can be switched off to test UC2-E4 | Only through TranslationAdapter (Adapter pattern) |
| eduroam authentication | – | Not used by UC2 (belongs to UC1) | EduroamAdapter planned for UC1 |

## 13. Performance Considerations

### 13.1 Performance Requirements

| Requirement | Target | Measured / design value |
|---|---|---|
| NFR-P.2 match results | < 3 s with 1,000 registered users | ≈ 2.2 ms average per search (UC2-T16, 60 searches) |
| NFR-P.1 page load | < 2 s | single 12 KB HTML page, no external resources |
| NFR-P.3 notifications | visible within 5 s | panel polls every 5 s; events are published synchronously |
| NFR-P.5 concurrent users | 100 in the demo environment | thread pool of max(4, 2 × CPU cores) threads; stateless service |

### 13.2 Scalability and Optimisation Strategies

- Matching is a linear scan, O(n) per search with n students, and each score is computed on small sets; results are cut to the top 10 before being returned.
- Profiles are loaded once and kept in an unmodifiable in-memory map: no file access during a search.
- Translations are cached in the adapter, so each text is translated once per language pair.
- For a larger deployment: pre-filter candidates in the database (type, department, language indexes), cache search results per student and criterion for a short time, and send notifications asynchronously through a queue so that a slow channel never delays a request.
- The stateless service allows several instances behind a load balancer once the repositories use a shared database.

## 14. Error Handling and Logging

### 14.1 Exception Management

All business errors extend **MatchingException**, which carries the SRS flow id and the HTTP status. The controller catches them in one place and returns a JSON error; unexpected errors are logged with their stack trace and returned as a generic 500 message, never as internal details.

| Exception | Flow / rule | HTTP | User-visible behaviour |
|---|---|---|---|
| IncompleteProfileException | UC2-E1 | 409 | Message asking to complete the profile |
| DuplicateRequestException | UC2-E3 | 409 | Toast: a request already exists |
| StudentNotFoundException | DV-01 | 404 | Error message |
| InvalidMatchRequestException | DV-02 … DV-04 | 400 | Error message with the reason |
| TranslationUnavailableException | UC2-E4 | (caught) | Original text with a notice; matching continues |
| IllegalArgumentException (bad parameter) | DV-05 | 400 | Error message |
| Exception in one observer | NFR-R.4 | – | Logged as SEVERE; other observers still notified |

UC2-E2 (no candidates) is not an error: the API returns an empty list with an explanatory message.

### 14.2 Logging Mechanisms

- java.util.logging with one-line format *date time LEVEL [logger] message*, to the console and to rotating files *logs/matching-%g.log* (5 × 1 MB). The level is set with the LOG_LEVEL variable (default INFO).
- INFO: start-up, number of students loaded, business errors with their flow id. FINE: every HTTP request with status and duration, every search with its number of candidates and time.
- WARNING: skipped data lines (DV-06), translation failures (UC2-E4). SEVERE: failing observers and unexpected errors, with stack trace.
- Audit trail: **MatchAuditLogObserver** writes one line per match event (type, request id, students, criterion, score, status) to the logger *matching.audit* – an example of adding a feature through the Observer pattern without changing the service.

## 15. Design for Testability

- **Dependency Injection:** every service receives its repositories, publisher, registry and Clock through the constructor. Tests build the same object graph with fresh in-memory repositories and a fixed clock (class Fixture), so tests are deterministic and independent.
- **Interfaces at every boundary:** strategies, observers, the translation service and the repositories can be replaced by test doubles (e.g. a failing observer in UC2-T13, a new strategy in UC2-T09, the mock API switched off in UC2-T11/T12).
- **No Singletons or static state** in the business code, so nothing leaks between tests.
- **Dependency-free test kit:** a small runner with JUnit-like assertions (testkit package) prints each test with its id, duration and covered requirements; the build fails if any test fails.
- **End-to-end test:** UC2-T18 starts the real web server on a free port and calls the API over HTTP.

| Test | Purpose | Covers |
|---|---|---|
| UC2-T01–T03 | Each ConcreteStrategy in isolation (exact scores) | UC2-S5, FR-2.1, FR-2.2 |
| UC2-T04 | Ranking, default filter and exclusion rules for all criteria | AT-UC2-01, UC2-S1–S6, FR-2.5 |

| Test | Purpose | Covers |
|---|---|---|
| UC2-T05 / T06 | Incomplete profile / no candidates | UC2-E1 / AT-UC2-03, UC2-E2 |
| UC2-T07 | Request accepted → confirmed, both notified | AT-UC2-02, UC2-S7–S9 |
| UC2-T08 | Duplicate, self, blocked and wrong-responder requests | UC2-E3, DV-02–DV-04 |
| UC2-T09 | New strategy used without changing the service | Strategy metric |
| UC2-T10 | Rejected candidate hidden for that criterion only; privacy | UC2-A3 |
| UC2-T11 / T12 | Adapter conversion, cache, error mapping / fallback text | UC2-A2, UC2-E4 |
| UC2-T13 | Observer attach/detach and fault isolation | Observer metric, NFR-R.4 |
| UC2-T14 / T15 / T17 | Filters / malformed data / JSON escaping | FR-2.3 / DV-06 / API safety |
| UC2-T16 | Performance with 1,000 students | NFR-P.2 |
| UC2-T18 | Full HTTP flow and error mapping | AT-UC2-01/02, API |

Result of the last run: **18 tests, 18 passed, 0 failed** (about 0.8 s).

# 16. Deployment and Installation Design

## 16.1 Environment Configuration

| Setting | Default | How to change |
|---|---|---|
| Java | JDK 17 or newer to build, JRE 17+ to run | – |
| HTTP port | 8080 | first program argument or PORT variable |
| Student data file | data/students.csv (packaged copy if missing) | second argument or STUDENTS_FILE variable |
| Log level | INFO | LOG_LEVEL variable (e.g. FINE) |
| Log files | logs/ in the working directory | – |

## 16.2 Packaging and Dependencies

```
# Linux / macOS                          # Windows
./build.sh        # compile + tests + jar   build.bat
./build.sh run    # and start the server    build.bat run
java -jar out/matching-uc2.jar 8080         # then open http://localhost:8080
```

- Output: *out/matching-uc2.jar* (≈ 75 KB) with the classes, the web page and a copy of the demo data; Main-Class com.sangreyoro.matching.MatchingApplication.
- Third-party dependencies: **none**. The build compiles with *--release 17* and treats all compiler warnings as errors.
- In production the server would run behind a reverse proxy that provides HTTPS (NFR-S.1).

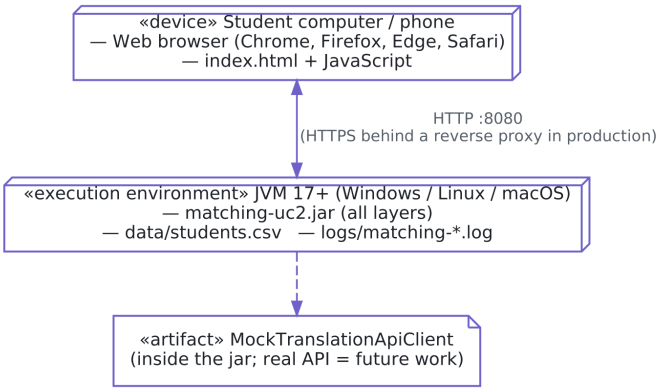



*Figure 15 – Deployment diagram*

## 17. Change Log

| Date | Version | Change | Author |
|---|---|---|---|
| 19 Oct 2026 | 0.1 | UC2 specified in the SRS (PA1) with Strategy as its pattern | Carlos Molina Perez |
| 22 Oct 2026 | 0.2 | UC2 + Strategy presented at the Design Pattern Workshop; Observer (notifications) and Adapter (translation) added to the scope of the implementation | Team |
| 06 Nov 2026 | 1.0 | Implementation of UC2 in three layers, 18 automated tests, web UI in three languages; Software Design Document; TASK_MATRIX.md and README updated | Team (see section 19) |

## 18. Future Work / Open Issues

| # | Item | Type |
|---|---|---|
| 1 | Replace the CSV and in-memory repositories with a relational database using the ER model of section 8.1 (only the Data Access Layer changes). | Future work (PA3) |
| 2 | Connect a real translation provider by writing a second adapter; add a timeout and retry policy. | Future work |
| 3 | Server-side messages (score reasons, notifications) are in English; localise them like the interface. | Open issue |
| 4 | Real authentication (UC1 / eduroam) instead of choosing the student in the demo list. | Open issue |
| 5 | Combined criteria (e.g. language + hobbies) as a composite strategy with weights. | Future work |
| 6 | Asynchronous notification delivery (queue) and e-mail / push observers. | Future work |
| 7 | Migrate the test kit to JUnit 5 when the build uses Maven (assertion names are already compatible). | Future work |

## 19. Task Matrix

| Task | Responsible |
|---|---|
| Sections 1–5: introduction, overview, context, features, assumptions | Carlos Molina Perez |
| Section 6: architectural design and rationale; architecture diagram | Alberto Alvarez Polo |
| Section 7: component design, component and sequence diagrams | Alejandro Blas Piris |
| Section 8: data design (ER diagram, storage, DFD, validation rules) | Carlos Molina Perez |
| Section 9: design patterns, class diagrams and metrics | Alejandro Blas Piris |
| Section 10: implementation notes | Carlos Molina Perez |
| Sections 11–12: user interface design and external interfaces | Alberto Alvarez Polo |
| Sections 13–16: performance, error handling and logging, testability, deployment | Carlos Molina Perez |
| Sections 17–18: change log, future work | Carlos Molina Perez |
| Review of the whole document | Alberto Alvarez Polo, Alejandro Blas Piris |
| Integration and final formatting | Carlos Molina Perez |